Binary Analysis and Patching Techniques

Tom Chothia

The aim of this part of the module is to develop an understanding of how dynamic methods can be used to analyse running binaries and how to control how they execute. The exercise game binary is not "stripped", meaning that all of the original object, method and variable names are still there. Analysing a "stripped" binary uses the same techniques but is harder because there are no descriptive names to help the reverse engineering process.

1 Static Analysis and Patching

The assembly of a program can be examined in tools such as IDA, gdb or Ghidra. Directly reading the assembly code is the best way to work out exactly what a binary is doing, but it's also the most time consuming. Once a binary has been understood, it can be directly altered with a hex edit or with tools like IDA. To edit a binary in IDA, we use the patch program options from the edit menu. For example, in lectures I patched the Example Game by first going to the `Player::GetSprintMultiplier` method. I clicked on the "mov rax 3" command, then edit > patch program > change bytes. I'm then presented with the raw bytes for the command "mov rax 3": 48 B8 03 00 00 00 00 00 00 00 and I can change 03 to FF, I can then see in IDA that the command has become "mov rax FF". I then go to Edit > patch program > apply changes to input file, to write this change to disk. This example gave me a big speed increase in the game.

Patching values in the exercise game is a little more complex because they are floats rather than ints. For these you will need to search online to find an online hex to float converter.

Advanced binaries may check their own code and even require it to be signed, which makes direct editing much harder (because you will also need to find and edit the part of the code that checks the code signing). On MAC you will need to re-sign any patched binary. As described in the exercise readme file.

2 Debugging with GDB

For the demos I use the peda extension for gdb `https://github.com/longld/peda`.

To debug a running process with gdb in Linux, you can either start the process with gdb or first run the process and then type `ps -A` in a terminal to get the process number, then type `sudo gdb -p <process number>`, to connect gdb to a running process. You break a running process by pressing control-C, and you can view assembly and set breakpoints in gdb (use the command c to make the process continue).

You can use gdb commands such as info functions, or info variables to see the game data. Importantly, you can use `ptype X'` to print all of the object types used in the game.

You can print variables, using * for pointer dereferences (in the normal C way), and casting values to particular types. E.g. for the exercise game:

```
print* GameWorld
```

prints the memory that GameWorld points to.

Gdb can also print out the type information of objects in member, e.g.,:

```
ptype Player
```

2.1 Breaking on functions in gdb

If you find an interesting looking function in IDA you can set a break point on it in gdb, e.g.,

```
break Player::GetJumpHoldTime
```

N.B. as the code we are looking at is dynamically linked the gdb will put a breakpoint on the real function and also on the pointer that links to it (in the "Procedure Linkage Table"). We do not want to break on this link function. So we type `info breakpoints` to view where breakpoints were set and disable the breakpoint marked with "@plt" e.g., `disable 1.1`.

After setting a breakpoint type `c` to make the programming continue.

Gdb then breaks on the GetJumpHoldTime function. You should be able to see the same code in gdb as you saw when looking at the function in IDA.

As this is a method of the player object "this" now equals the player object, so we can view it and set values using the following code.

```
set $playerObj = this
print * (Player*) $playerObj
get $playerObj->m_jumpHoldtime
set $playerObj->m_jumpHoldtime = 60
```

N.B. if you cast memory to be the wrong type or dereference the wrong variables, you might get an error saying that the memory is not accessible, or you might get values that appear random as they are not of the type that gdb thinks they are. There are many excellent gdb tutorials online that you can use to find more gdb commands.

Games with advanced anti-cheat, and some other binaries, may try to detect if a debugger has been attached to a process and kick or ban the user. You can find more details on that here: `https://www.cs.bham.ac.uk/~tpc/Papers/AntiCheat2024.pdf`

3 Hooking Functions

Hooking functions is a technique that attaches your own code to run whenever a function is called. It is a very useful binary analysis method. E.g., we can "hock" the GetJumpHoldTime method and make is run the following code:

```
break Player::GetJumpHoldTime
commands $bpnum
  set $pc = $pc + 8
  set $xmm0.v4_float = {60, 0, 0, 0}
  continue
end
```

The first line sets a breakpoint. `commands $bpnum` tells gdb to run the following commands when the breakpoint is hit. `set $pc = $pc` 8+ moves the program counter/instruction pointer forward by 8 bytes, skipping the real code write to xmm0. The `set $xmm0.v4_float = {60, 0, 0,` sets the register xmm0 (the register used to return floating point values) to 60. Therefore this function now returns our chosen value of 60.0, rather than the intended value.

When investigating other applications hooking is a very useful method to print out cryptographic keys, remove encryption, or log useful information. An excellent, more advanced framework for hooking functions is Frida: `https://frida.re`.

4 Memory Layout Analysis with Cheat Engine

The Cheat Engine tool on Windows lets us find arbitrary values in memory. It's great for finding player values in a game, but it is also really good for finding keys and other cryptographic information in a programs memory. There is a download link for cheat engine on the module website, and it comes with a great tutorial (Help > Tutorial) that I highly recommend.

We use cheat engine by starting the target process and cheat engine and then clicking on the attach button (top left) and attaching cheat engine to the target value. We can now scan for values in memory. N.B. you will have to guess the type of the value you are looking for, is it a 4-byte int? a 2-byte int? A double? A float? If you guess wrong then you will not find the value.

Find the value in memory by repeatedly scanning, and changing the value and scanning again. Once you have found it double click it to add the memory address to your list and give it a useful name. This list can be saved and reloaded.

However, due to ASLR and differences in how program runs each time it is started, the interesting values will be in different places in memory each time you start the program. Therefore you need to find a "pointer path" from the base of the program or dll to the value we want.

We find pointer paths by right clicking the address from our list and selecting, Pointer scan for this address. This will take a while and find all pointer paths that go to this

address. To find out which pointer paths work between process restarts, kill the game process, restart it and reattached Cheat Engine. Then rescan the pointers, by going to rescan memory and entering the expected value, or finding the memory address again and entering that. The points that correctly point to the memory you want after each restart can then be saved to your address list. Click on the address to see the pointer path that you can use in a dll, as described below.

Cheat Engine has a lot more functionality, such as a debugger, and the ability to explore memory regions, that I highly recommend you experiment with.

5 DLL injection in Windows

DLL injection in Windows lets you run code inside the target process. The injected code will have read and write access to all of the memory of the target process, i.e., values that would not normally be displayed to the user can be found and additional behaviours can be added. DLLs can be written with Visual Studio and injected with "extreme injector", as well as many other tools (N.B. injection tools and code may be flagged up by your anti-virus, in which case you will need to temperately disable this). N.B. injecting DLLs is not required for any of the exercises, but could be useful for, e.g,, flying or teleporting.

You can create a DLL in Visual Studio by creating a new project and selecting Dynamically Linked Library with exports. This will create a project with a stub dllmain.cpp file. The commented example code from lectures is on the module canvas page. You can compile this by going to Build > Build Solution, which will give you a dll file in the Debug directory. This can be injected into the target process by selecting the target process and the DLL in extreme injector and clicking inject.

The most common error when doing this is to get memory addresses wrong, leading crashes due to memory access violations, or getting seemingly random values back. The best way to debug this is to print as much as possible to the console and compare the address you get with what you see in Cheat Engine.

6 What you need to know for the exam

For the exam I want you to understand the concepts and what is possible in theory, particularly:

- Be aware that it is possible to patch a binary file to change the way it functions.
- Be aware that It is possible to search the memory of a running process for values.
- You should understand what "hooking a function" means and when it might be useful.
- You should understand its possible to run your own code that shares memory with the program you are testing.

For the exam you do not need to understand how to do this in practice. However, some of these methods may be useful for the continuous assessment.